

Prompt de desarrollo: Aplicación gamificada para aprender la Terminal / CLI

1. Resumen del proyecto

Desarrollar una aplicación educativa en formato de **juego** que enseñe a estudiantes de **12 a 13 años** a usar la **línea de comandos (CLI)** de forma guiada, progresiva y entretenida. La app debe combinar una interfaz llamativa tipo videojuego con una **terminal simulada** que imite el aspecto clásico de las consolas de los sistemas operativos actuales (fondo oscuro, texto monoespaciado, estética "hacker retro").

El objetivo pedagógico es que el estudiante aprenda, mediante misiones y recompensas, a manejar comandos **básicos y genéricos**, es decir, que no dependan de un sistema operativo específico (Windows, macOS o Linux), sino que representen la lógica general de cómo funciona una terminal.

2. Público objetivo

- Edad: 12-13 años (primer contacto con la línea de comandos).
 - Sin conocimientos previos de programación ni de sistemas operativos.
 - Familiarizados con videojuegos, apps móviles y mecánicas de gamificación (niveles, puntos, insignias).
-

3. Objetivo pedagógico

Que el estudiante, al finalizar los distintos niveles/misiones, sea capaz de:

1. Entender qué es una terminal y para qué sirve.
 2. Moverse entre carpetas/directorios.
 3. Crear, ver, mover, copiar y eliminar archivos y carpetas.
 4. Entender la estructura jerárquica de un sistema de archivos.
 5. Leer mensajes de error básicos y corregir errores de sintaxis simples.
 6. Perder el miedo a la "pantalla negra" y ganar confianza para seguir explorando por su cuenta.
-

4. Concepto general del juego

- Es un **juego de progresión por misiones** (estilo "aventura" o "isla de niveles"), donde cada misión introduce uno o más comandos nuevos.
- Cada misión tiene: una breve historia/contexto narrativo, un objetivo claro, instrucciones guiadas paso a paso, y una recompensa al completarla.
- El estudiante interactúa principalmente con una **terminal simulada** (no una terminal real del sistema, por seguridad), donde escribe comandos y recibe respuestas simuladas coherentes con lo que ocurriría en una terminal real.

- Alrededor de la terminal hay una interfaz de juego (mapa de niveles, personaje/mascota guía, sistema de puntos, insignias, sonidos, animaciones).
-

5. Mecánicas de juego

5.1 Estructura de misiones

- Las misiones están organizadas en **mundos o capítulos temáticos** (por ejemplo: "La isla de los archivos perdidos", "El laberinto de carpetas", etc. — el equipo de arte/narrativa puede proponer un hilo conductor).
- Cada misión debe incluir:
 - Un objetivo narrativo breve (2-3 líneas).
 - Un objetivo técnico concreto ("crea una carpeta llamada **tesoro**").
 - Pistas/ayudas progresivas si el estudiante se traba (sistema de 2-3 niveles de ayuda antes de mostrar la solución).
 - Validación automática del comando ingresado (o de su resultado) para saber si la misión se completó.
 - Un mensaje de éxito y una recompensa.

5.2 Tipos de misiones (variedad)

- **Misión guiada:** el juego indica exactamente qué comando escribir, paso a paso (para las primeras veces que se introduce un comando).
- **Misión de práctica libre:** se da un objetivo y el estudiante debe deducir el comando (con pistas disponibles).
- **Misión de detective de errores:** se le muestra un comando mal escrito o un mensaje de error, y debe identificar y corregir el problema.
- **Misión de desafío cronometrado (opcional, para variar el ritmo):** completar una secuencia de comandos ya aprendidos contra el tiempo.
- **Misión "boss" de fin de mundo:** combina todos los comandos aprendidos en ese bloque en un desafío final más largo.

5.3 Sistema de recompensas

- Puntos de experiencia (XP) por misión completada.
- Insignias/logros por hitos (ej. "Primer comando", "Maestro de carpetas", "Sin errores en 5 misiones seguidas").
- Elementos cosméticos desbloqueables (skins de terminal, colores de texto, avatar/mascota, sonidos).
- Barra de progreso visible por mundo y progreso general.
- Mensajes de aliento ante errores (nunca mensajes punitivos o desmotivadores).

5.4 Progresión de dificultad

- Empezar por comandos únicos y aislados, y progresivamente combinarlos en secuencias más largas.

- Ir soltando el nivel de guía (de "guiado paso a paso" a "resolvé con pistas" a "resolvé solo").
 - El diseño debe permitir **agregar nuevos mundos y comandos más avanzados en el futuro** sin rehacer la arquitectura (ver sección 8).
-

6. Comandos a incluir en esta primera versión

Usar comandos **genéricos y conceptuales**, no atados a un sistema operativo particular. Se sugiere trabajar con una sintaxis simplificada e inventada por el juego (tipo "pseudo-CLI") que represente la lógica común a Windows/macOS/Linux, en lugar de elegir la sintaxis real de uno solo (evitando así enseñar desde el inicio una sintaxis que no sirva en otro sistema). El equipo de contenido puede ajustar los nombres exactos, pero deben cubrir estas acciones básicas:

1. Ver en qué carpeta estoy parado (equivalente a "dónde estoy").
2. Listar el contenido de una carpeta (ver qué hay adentro).
3. Moverme a otra carpeta (entrar/salir de carpetas).
4. Crear una carpeta nueva.
5. Crear un archivo nuevo.
6. Ver el contenido de un archivo de texto simple.
7. Renombrar un archivo o carpeta.
8. Mover un archivo de una carpeta a otra.
9. Copiar un archivo.
10. Eliminar un archivo o carpeta.
11. Pedir ayuda / ver qué comandos existen.
12. Limpiar la pantalla de la terminal.

No incluir en esta versión: permisos de usuario, redes, procesos, variables de entorno, scripting, ni comandos destructivos irreversibles sin confirmación. Todo debe ser reversible dentro del entorno simulado (por ejemplo, con un botón de "reiniciar misión").

7. Requisitos de interfaz (UI/UX)

7.1 Terminal simulada

- Debe imitar visualmente una consola real: fondo oscuro (negro o gris muy oscuro), tipografía monoespaciada, cursor parpadeante, texto en verde/blanco/ámbar (configurable como skin desbloqueable).
- Debe mostrar el prompt de forma clara (ej. `usuario@isla:~$`).
- Autocompletado visual opcional y resaltado de sintaxis para ayudar a la lectura.
- Mensajes de error simulados deben ser realistas pero traducidos a un lenguaje amigable para la edad (sin perder el formato real de un error de consola).
- Historial de comandos navegable (flechas arriba/abajo), igual que en una terminal real, para que el aprendizaje transfiera bien a una terminal de verdad.

7.2 Interfaz de juego (alrededor de la terminal)

- Estética colorida, amigable y "gamer", contrastando con el negro de la consola (para que la terminal se sienta como "la herramienta especial" dentro de un mundo más vistoso).
- Mapa de progreso/niveles visual (isla, laberinto, galaxia, etc., a definir por diseño).
- Mascota o personaje guía que acompaña, da contexto narrativo y feedback.
- Panel de ayuda/pistas accesible en todo momento.
- Sonidos y animaciones de éxito/error, sutiles y no intrusivos.
- Indicadores claros de progreso (XP, insignias, nivel actual).

7.3 Accesibilidad y usabilidad

- Tipografía legible, tamaños ajustables.
 - Alto contraste disponible.
 - Textos breves y en lenguaje claro, adecuado para 12-13 años (evitar jerga técnica sin explicar).
 - Soporte para que un docente/tutor pueda ver el progreso del estudiante (ver sección 9, opcional según alcance).
-

8. Requisitos técnicos y de arquitectura

- La terminal debe ser **completamente simulada** (sandbox), sin ejecutar comandos reales del sistema operativo del dispositivo, por razones de seguridad.
 - Diseñar el sistema de comandos y misiones de forma **modular y configurable** (por ejemplo, mediante archivos de configuración o un motor de contenido), para poder agregar fácilmente:
 - Nuevos comandos.
 - Nuevos mundos/misiones.
 - Nuevos niveles de dificultad.
 - Guardar el progreso del estudiante (local y/o en la nube, a definir según alcance del proyecto).
 - Definir plataforma objetivo: **web, escritorio o móvil** (a confirmar con el equipo, ya que condiciona el stack tecnológico).
 - Rendimiento adecuado para uso en aulas (equipos con recursos limitados si aplica).
-

9. Alcance de esta primera versión (MVP)

Incluir:

- 1 mundo introductorio completo (tutorial de "qué es la terminal").
- 2-3 mundos temáticos con los 12 comandos listados en la sección 6.
- Sistema de misiones guiadas + práctica libre + detective de errores.
- Sistema de XP, insignias y progreso visual.

- Terminal simulada con historial de comandos y mensajes de error amigables.

Fuera de alcance por ahora (pero a tener en cuenta en el diseño para no bloquear el futuro):

- Comandos avanzados (permisos, redes, scripting).
 - Modo multijugador o competitivo.
 - Panel para docentes con métricas de toda una clase.
-

10. Entregables esperados del equipo de desarrollo

1. Propuesta de arquitectura técnica y stack sugerido.
 2. Prototipo navegable de la terminal simulada + una misión completa de ejemplo.
 3. Documento de diseño de niveles/misiones (o herramienta para cargarlos).
 4. Build funcional del MVP descrito en la sección 9.
 5. Documentación para que el contenido pedagógico (misiones, textos, comandos) pueda ser editado/ampliado sin tocar código, en la medida de lo posible.
-

11. Tono y estilo general

- Amigable, motivador, nunca punitivo ante los errores (los errores son parte del aprendizaje y deben celebrarse como "pistas", no como fracasos).
- Estética que combine lo "gamer" con lo "tech/hacker", pensada para captar el interés de esta franja etaria sin resultar infantil ni intimidante.